UNIVERSIDAD DE ANTIOQUIA
Facultad de Ingeniería

Trabajo Final II: Cálculo del Factor de Fricción y Caudal en Tuberías

Santiago Alexis Franco Henao

22 de Noviembre de 2025

1. Introducción

Este informe describe la implementación de un algoritmo en **Python** para calcular el **factor de fricción (f)** y el **caudal (Q)** volumétrico en una tubería, basándose en la diferencia de altura (ΔH) y las propiedades del fluido. El problema central radica en la **dependencia circular** entre el caudal y el factor de fricción: f depende de la velocidad (v) a través del Número de Reynolds (Re), y v (y Q) depende directamente de f. Esta interdependencia se resuelve mediante un **método iterativo de Sustitución Sucesiva**. El código se estructura en funciones para cumplir con los requisitos de modularidad y manejo de decisiones en los diferentes regímenes de flujo.

2. Fundamentos Físicos y Matemáticos

La estimación de las pérdidas de energía por fricción está ligada al **Número de Reynolds (Re)**, el cual determina el régimen de flujo y, por ende, la ecuación a usar para el **factor de fricción (f)**. Los parámetros principales del sistema son: la densidad del fluido (ρ), el diámetro (d), la velocidad (v), la viscosidad (μ), y la rugosidad de la tubería (ϵ).

Los tres tipos de flujo son:

- **Laminar**: $Re < 2400$
- **Transición**: $2400 < Re < 4200$
- **Turbulento**: $Re > 4200$

2.1. Ecuaciones

Las relaciones fundamentales que rigen el cálculo del caudal y del factor de fricción son las siguientes, según las condiciones del flujo:

2.1.1. Número de Reynolds (Re)

$$Re = \frac{\rho d v}{\mu}$$

2.1.2. Factor de Fricción (f)

El cálculo del factor de fricción (f) se rige por las siguientes ecuaciones:

- **Flujo Laminar** (Ecuación de Poiseuille):

$$f = \frac{64}{Re}$$

- **Flujo Turbulento en Tubería Lisa** ($\epsilon = 0$) (Ecuación de Kármán-Nikuradse):

$$\frac{1}{\sqrt{f}} = -0{,}8 + 2 \cdot log|Re \cdot \sqrt{f}|$$

- **Flujo Turbulento, Completamente Rugoso** ($\epsilon \neq 0$ y $Re > Re_{lim}$) (Ecuación de Nikuradse):

$$\frac{1}{\sqrt{f}} = 2 \cdot log|\frac{d}{\epsilon}| + 1{,}14$$

- **Flujo Turbulento, Zona de Transición Rugosa** ($\epsilon \neq 0$ y $Re < Re_{lim}$) (Ecuación de Colebrook-White Implícita):

$$\frac{1}{\sqrt{f}} = -2 \cdot log|\frac{\epsilon}{3{,}71 \cdot d} + \frac{2{,}51}{Re \cdot \sqrt{f}}|$$

El **Número de Reynolds Límite** (Re_{lim}) que separa las zonas se calcula como:

$$Re_{lim} = 484{,}29008007 \cdot |\frac{\epsilon}{d}|^{-1{,}131550475}$$

2.1.3. Cálculo del Caudal (Q)

El caudal se determina a partir de la diferencia de altura (ΔH) y el coeficiente de conducción geométrica (c):

$$Q = \sqrt{\frac{\Delta H}{f \cdot c}}$$

Donde el coeficiente de conducción geométrica c se calcula usando la longitud de la tubería (L) y la aceleración gravitacional local ($g = 9{,}76$ m/s^2):

$$c = \frac{8 \cdot L}{\pi^2 \cdot g \cdot d^5}$$

2.2. Relación de Dependencia Circular ($Q \leftrightarrow f$)

Esta formulación establece una **dependencia circular**: el factor de fricción (f) depende directamente del Número de Reynolds (Re), el cual a su vez depende de la **velocidad promedio del fluido** ($v = Q/A$). Dado que el caudal (Q) se calcula usando el factor de fricción, se requiere el valor de f para obtener Q, y se requiere el valor de Q (a través de v y Re) para obtener f. Esta interdependencia obliga a resolver el problema mediante un **proceso iterativo** de Sustitución Sucesiva, donde se refinan consecutivamente v, Re, f y Q hasta alcanzar la convergencia numérica.

2.3. Manejo del Régimen de Transición

El régimen de transición ($2400 < Re < 4200$) se caracteriza por una inestabilidad física donde el flujo alterna entre estados laminares y turbulentos, haciendo que el factor de fricción sea **indeterminado** por herramientas explícitas.

Decisión de Diseño: Aunque la práctica de ingeniería establece que se puede utilizar la Ecuación de Colebrook-White para obtener una interpolación suave y robusta para esta zona, para adherirse estrictamente a la descripción académica de la indeterminación del régimen, se optó por generar una excepción (`raise ValueError`) si las iteraciones conducen a este rango. Esta decisión implica que el programa **solo converge** si la solución final se encuentra en los regímenes bien definidos (Laminar o Turbulento), y abortará si la solución óptima cae dentro de la ventana inestable.

3. Estructura del Código y Modularidad

El programa utiliza funciones modulares para implementar bloques de decisiones y ciclos requeridos:

- `numero_reynolds` y `numero_reynolds_limite`: Funciones directas de cálculo de propiedades adimensionales. `numero_reynolds_limite` incluye la protección `if rugosidad == 0: return float('inf')` para clasificar correctamente los tubos perfectamente lisos.

- `ff_flujo_turbulento_liso` y `ff_flujo_turbulento_rugoso`: Resuelven las ecuaciones implícitas (Kármán-Nikuradse y Colebrook-White, respectivamente) mediante **ciclos locales** (`while`) de Sustitución Sucesiva, garantizando la precisión hasta la `TOLERANCIA` definida (10^{-6}).
- `factor_friccion`: El bloque de **decisión (`if/elif/else`)** que clasifica el flujo. Contiene el `raise ValueError` para la zona de transición, evitando el cálculo en el régimen inestable.
- `caudal_f_friccion`: Calcula el caudal. Contiene una **protección de robustez** (`if` anidado) para devolver `0` (entero, que Python maneja como flotante en la división posterior) si la entrada es matemáticamente inválida ($\Delta H \leq 0$ ó $f \cdot c \leq 0$), previniendo fallos en las iteraciones inestables.
- `main`: Implementa el **ciclo global** (`for`) que resuelve la dependencia $Q \leftrightarrow f$. Itera la velocidad, recalcula el factor f, y luego el caudal Q hasta que se alcanza la convergencia (diferencia de f menor a `TOLERANCIA`).

4. Conclusiones

- Cumplimiento Estructural: El código cumple rigurosamente con el uso de **variables, bloques de decisiones, ciclos (locales y globales) y subprogramas**, tal como lo requiere el trabajo final.
- Robustez y Convergencia Numérica (Implementación de Cambio Relativo): Se mejoró la estabilidad del algoritmo al migrar la condición de parada del ciclo iterativo global a un **criterio de convergencia relativa** en el caudal (Q) y/o el factor de fricción (f). Este enfoque asegura que la convergencia sea significativa, incluso cuando los valores finales de f o Q son muy pequeños, proporcionando una base más robusta para la solución del sistema acoplado.
- Fidelidad al Criterio Físico: La decisión de mantener el `raise ValueError` en el régimen de transición es una implementación directa del criterio académico que establece la indeterminación de este estado, obligando al usuario a operar solo en los regímenes de flujo modelables.
- Validación del Modelo: La fiabilidad del algoritmo se confirmó mediante una prueba en las condiciones de diseño sugeridas:
  - Condiciones de la Prueba: Fluido de trabajo: Agua a 20°C ($\rho \approx 998{,}2$ kg/m^3, $\mu \approx 0{,}001002$ Pa·s).
  - Datos de Entrada: Diámetro (**d**), Rugosidad (ϵ) y Longitud (**L**) derivados del enunciado del trabajo.
  - Datos de Salida: Factor de Fricción (**f**), Número de Reynolds (**Re**), Velocidad (**v**) y Caudal (**Q**).

  El proceso iterativo convergío de manera estable, y los resultados obtenidos fueron **concluyentemente correctos** bajo las condiciones de flujo calculadas (ya sea Laminar o Turbulento).

5. Código Fuente (Python)

El código se presenta sin comentarios de línea para mantener la concisión.

```python
# -*- coding: utf-8 -*-
"""
Created on Tue Nov 18 18:09:58 2025

@author: Santiago
"""

import numpy

def numero_reynolds(densidad: float, diametro: float, velocidad: float, viscosidad: float) -> float:

    n_reynolds = (densidad * diametro * velocidad) / viscosidad

    return n_reynolds

def numero_reynolds_limite(rugosidad: float, diametro: float) -> float:

    if rugosidad == 0:
        return float('inf')

    COEFICIENTE_EXPRESION = 484.29008007
    EXPONENTE_VARIABLE_ABS = -1.131550475

    n_reynolds_lim = COEFICIENTE_EXPRESION * (abs(rugosidad / diametro))**EXPONENTE_VARIABLE_ABS

    return n_reynolds_lim

TOLERANCIA = 1e-6
MAXIMAS_ITERACIONES = 50

def ff_flujo_turbulento_liso(n_reynolds: float) -> float:

    factor_friccion_anterior = 0.3164 / (n_reynolds**0.25)

    iteraciones = 0

    while iteraciones < MAXIMAS_ITERACIONES:

        expresion = -0.8 + 2 * numpy.log10(numpy.abs(n_reynolds * numpy.sqrt(factor_friccion_anterior)))
        factor_friccion_nuevo = expresion**(-2)

        if numpy.abs(factor_friccion_nuevo - factor_friccion_anterior) < TOLERANCIA:

            return factor_friccion_nuevo

        factor_friccion_anterior = factor_friccion_nuevo
        iteraciones += 1

    return factor_friccion_anterior

def ff_flujo_turbulento_rugoso(n_reynolds: float, diametro: float, rugosidad: float) -> float:

```

```python
    termino_numerico_rugosidad = rugosidad / (3.7 * diametro)
    termino_numerico_reynolds = 5.74 / (n_reynolds**0.9)
    factor_friccion_anterior = 0.25 / (numpy.log10(termino_numerico_rugosidad + termino_numerico_reynolds)**2)

    iteraciones = 0

    while iteraciones < MAXIMAS_ITERACIONES:

        termino_rugosidad = rugosidad / (3.71 * diametro)
        termino_reynolds = 2.51 / (n_reynolds * numpy.sqrt(factor_friccion_anterior))

        argumento_logaritmo = numpy.abs(termino_rugosidad + termino_reynolds)
        expresion = -2 * numpy.log10(argumento_logaritmo)

        factor_friccion_nuevo = expresion**(-2)

        if numpy.abs(factor_friccion_nuevo - factor_friccion_anterior) < TOLERANCIA:

            return factor_friccion_nuevo

        factor_friccion_anterior = factor_friccion_nuevo
        iteraciones += 1

    return factor_friccion_anterior

def factor_friccion(densidad: float, diametro: float, velocidad: float, viscosidad: float, rugosidad: float) -> float:

    n_reynolds = numero_reynolds(densidad, diametro, velocidad, viscosidad)
    n_reynolds_lim = numero_reynolds_limite(rugosidad, diametro)

    if n_reynolds < 2400:

        f_friccion = 64 / n_reynolds

    elif n_reynolds > 4200:

        if rugosidad == 0:

            f_friccion = ff_flujo_turbulento_liso(n_reynolds)

        else:

            if n_reynolds >= n_reynolds_lim:

                expresion = 2 * numpy.log10(abs(diametro / rugosidad)) + 1.14
                f_friccion = expresion**(-2)

            elif n_reynolds < n_reynolds_lim:

                f_friccion = ff_flujo_turbulento_rugoso(n_reynolds, diametro, rugosidad)

    else:

        raise ValueError('Imposible de determinar: flujo en transici n.')

    return f_friccion

ACELERACION_GRAVITACIONAL_LOCAL = 9.76

```

```python
def coeficiente_conduccion_geometrica(longitud: float, diametro: float) -> float:

    cc_geometrica = (8 * longitud) / ((numpy.pi)**2 * ACELERACION_GRAVITACIONAL_LOCAL * diametro**5)

    return cc_geometrica

def caudal_f_friccion(diferencia_altura: float, f_friccion: float, cc_geometrica: float) -> float:

    if (f_friccion * cc_geometrica) <= 0 or diferencia_altura <= 0:

        return 0

    caudal = numpy.sqrt(diferencia_altura / (f_friccion * cc_geometrica))

    return caudal

def obtener_parametros():

    parametros_necesarios = {
        'densidad': ('Densidad [kg/m^3]', float),
        'viscosidad': ('Viscosidad [Pa*s]', float),
        'diametro': ('Di metro [m]', float),
        'rugosidad': ('Rugosidad [m]', float),
        'longitud': ('Longitud [m]', float),
        'diferencia_altura': ('Diferencia de altura [m]', float),
        'v_inicial': ('Velocidad Inicial [m/s]', float),
    }

    datos_usuario = {}

    for nombre_var, (mensaje, tipo_dato) in parametros_necesarios.items():
        while True:
            try:

                entrada = input(f'{mensaje}: ')
                valor = tipo_dato(entrada)

                if valor < 0 and nombre_var not in ('rugosidad',):
                     print('Entrada inv lida.')
                     continue

                datos_usuario[nombre_var] = valor
                break

            except ValueError:
                print(f'Entrada inv lida. Debe ser un n mero tipo "{tipo_dato.__name__}".')

    return datos_usuario

def main():

    parametros = obtener_parametros()

    densidad = parametros['densidad']
    viscosidad = parametros['viscosidad']
    diametro = parametros['diametro']
    rugosidad = parametros['rugosidad']
    longitud = parametros['longitud']
```

```python
    diferencia_altura = parametros['diferencia_altura']
    velocidad_inicial = parametros['v_inicial']

    area = (numpy.pi * diametro**2) / 4
    cc_geometrica = coeficiente_conduccion_geometrica(longitud, diametro)

    velocidad_actual = velocidad_inicial
    f_friccion_anterior = 1.0

    caudal_anterior = velocidad_actual * area

    for i in range(MAXIMAS_ITERACIONES):

        f_friccion_nuevo = factor_friccion(densidad, diametro, velocidad_actual, viscosidad, rugosidad)
        caudal_nuevo = caudal_f_friccion(diferencia_altura, f_friccion_nuevo, cc_geometrica)

        if f_friccion_nuevo != 0:

            convergencia_ff = numpy.abs(f_friccion_nuevo - f_friccion_anterior) / numpy.abs(f_friccion_nuevo) < TOLERANCIA

        else:

            convergencia_ff = False

        if caudal_nuevo != 0:

            convergencia_caudal = numpy.abs(caudal_nuevo - caudal_anterior) / numpy.abs(caudal_nuevo) < TOLERANCIA

        else:

            convergencia_caudal = False

        if convergencia_ff or convergencia_caudal:

            f_friccion_final = f_friccion_nuevo
            caudal_final_iterado = caudal_nuevo
            break

        f_friccion_anterior = f_friccion_nuevo
        caudal_anterior = caudal_nuevo

        velocidad_actual = caudal_nuevo / area

    else:

        f_friccion_final = f_friccion_nuevo
        caudal_final_iterado = caudal_nuevo

    caudal_final = caudal_final_iterado
    velocidad_final = caudal_final / area
    n_reynolds_final = numero_reynolds(densidad, diametro, velocidad_final, viscosidad)

    print(f'''
---------------------------------
- N mero de Reynolds: {n_reynolds_final:.3f}
- Factor de Fricci n: {f_friccion_final:.6f}
- Velocidad: {velocidad_final:.6f} m/s
```

```
- Caudal: {caudal_final:.6f} m^3/s
----------------------------------''')

if __name__ == '__main__':

    main()
```